



PROJECT-BASED LEARNING

PHASE-I EVALUATION

**EcoGraph – Graph Based
Algorithmic Waste Management**

Team ID: DSCPP-III-2026-T186

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T186
Semester: 3rd
Section: A / D / H
Domain: Graph Algorithms (C++)

Team Members

1. Tejas Jain – 2028228 (SEC-A)
2. Yash Nigam – 2028289 (SEC-H)
3. Apoorv Negi – 2027640 (SEC-D)

Mentor

Navneet Rajput

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Municipal waste routes are static, ignoring the real shortest path between homes, bins, and the depot – wasting fuel and time. Hazardous waste (medical, chemical) is handled like ordinary waste despite needing immediate, restricted-zone pickup, and fixed schedules over- or under-service locations with different fill rates.

Motivation

- ▶ Fixed routes ignore real shortest paths in the city graph
- ▶ Hazardous waste in normal queues risks exposure delays
- ▶ Uniform schedules waste trips or under-serve fast bins

Target Users / Stakeholders

- ▶ Municipal waste collection departments
- ▶ Residents and commercial establishments served
- ▶ Public health – faster hazardous waste clearance

Objectives

Project Objectives

1. Model the city as a weighted graph (depot, homes, junctions, bins) tagged by zone type.
2. Compute optimal routes using Dijkstra's algorithm, with selectable Priority and FIFO strategies.
3. Detect and immediately dispatch hazardous waste via a zone-constrained shortest path avoiding residential/commercial areas.
4. Dynamically schedule collection frequency per location based on its fill-rate.
5. Expose all functionality through a REST API with live route animation on the frontend.

Key Objective: A persistent C++ backend that computes correct, zone-aware, fill-rate-driven collection routes and serves them as JSON to an animated web frontend.

Proposed Solution

Proposed Approach

1. City layout loaded from JSON config at startup
2. Dijkstra's algorithm (plain + zone-constrained) as the core routing engine
3. Priority/FIFO route builders + fill-rate scheduler
4. In-memory graph state (`unordered_map`)
5. REST API (JSON) + Cytoscape.js animation

A single persistent C++ server owns the graph, performs routing/classification/scheduling in-process, serves JSON to the JS frontend.

Key Features

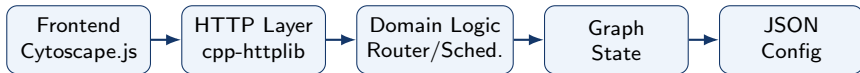
- ▶ Graph-based city model with zone tagging
- ▶ Shortest-path routing (Priority / FIFO toggle)
- ▶ Rule-based waste classification (keywords)
- ▶ Hazardous waste: immediate, constrained routing
- ▶ Dynamic collection frequency from fill-rate

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Graph (adjacency list), Queue (FIFO), Priority queue / min-heap, Hash map (<code>unordered_map</code>)	Graph model, Scheduler, Router
OOPs with C++	Classes for Graph, Node, Edge, Scheduler, Router; STL containers; encapsulated domain layer	Backend domain layer (<code>graph.hpp</code> , <code>scheduler.hpp</code> , <code>router.hpp</code>)

Integration: The city graph (DSA: adjacency list + hashing) is implemented as C++ classes (OOP: encapsulation, STL) exposing Dijkstra, constrained Dijkstra, FIFO/priority-queue-based routing, and scheduling as one cohesive backend.

System Architecture / Workflow



Major Components

- ▶ Graph model (Node, Edge, zone tags)
- ▶ Routing engine (plain + constrained Dijkstra)
- ▶ Classifier + Scheduler + REST controllers

Data / Control Flow

- ▶ Frontend calls /route, /classify, /schedule, /route/hazard
- ▶ Backend loads city.json once, mutates state in memory
- ▶ Response: JSON path/cost, animated on the graph canvas

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C++ (backend), JavaScript (frontend)
- ▶ HTTP: cpp-httplib (single-header)
- ▶ JSON: nlohmann/json (header-only)
- ▶ Frontend: HTML/CSS/JS + Cytoscape.js
- ▶ Build: CMake; Version Control: Git/GitHub

Development Methodology

1. Requirement Analysis (proposal finalized)
2. System & API Design (architecture, JSON contracts)
3. Phased Implementation (graph → algorithms → API → frontend)
4. Unit Testing (Dijkstra, classifier, scheduler)
5. Live Demo Evaluation

Header-only libraries keep the backend dependency-light; Cytoscape.js gives native graph rendering and animation.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Problem study and proposal finalized
- ▶ Full system architecture designed
- ▶ REST API contracts (5 endpoints) specified
- ▶ C++ class design for Graph, Router, Scheduler, Classifier
- ▶ Sample `city.json` graph and repo structure planned

Individual Contribution

Member	Role	Contribution
Tejas Jain	Backend Lead	Graph model, build system, routing modes, HTTP server, <code>/route</code> , <code>/graph</code> , animation
Apoorv Negi	Algorithms	Plain & constrained Dijkstra, hazard dispatch, <code>/route/hazard</code> , hazard styling
Yash Nigam	Data & Scheduling	City graph, waste classifier, scheduler, <code>/classify</code> , <code>/schedule</code> , rendering

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal, Architecture & API Design (done)
2. Phase-II: Graph engine, algorithms, REST API, frontend
3. Phase-III: Full demo, stretch goals (concurrency, persistence)

Expected Outcomes

- ▶ Working C++ backend + animated web frontend
- ▶ Verified shortest-path and constrained-route correctness
- ▶ Demonstrated fill-rate-driven dynamic scheduling
- ▶ Extensible to concurrency and persistent storage

References

1. Cormen et al. – *Intro. to Algorithms* (Dijkstra's Algorithm)
2. cpp-httpplib – github.com/yhirose/cpp-httpplib
3. nlohmann/json – github.com/nlohmann/json
4. Cytoscape.js – js.cytoscape.org

Thank You
Questions & Suggestions